Sublinear Algorithms for $(\Delta + 1)$ Vertex Coloring

Mohammad Hosein Rabiee Amir Hosein Karimzadegan
Mohammad Reza Ahmadi Behrad Sanei

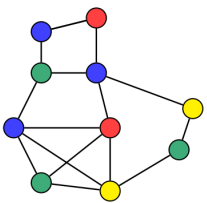
Department of Computer Science and Statistics
Faculty of Mathematics
K. N. Toosi University of Technology

Overview

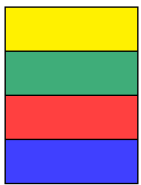
1. Graph Coloring
2. Sublinear Algorithms
3. Main Results
4. Palette Sparsification Theorem
5. A One-Pass $\tilde{O}(n)$ Space Streaming Algorithm
6. Maintaining Algorithm For Dynamic Streams
7. Proof Idea of the Palette-Sparsification Theorem
8. Related Works

Graph Coloring

- Assign a small number of colors to vertices of a graph $G(V, E)$ such that there no monochromatic edges (has the same color on both endpoints).
- A proper c-coloring of a graph $G(V, E)$:
  - assigns a color from the palette $\{1, 2, \ldots, c\}$ to all vertices V of G,
  - no monochromatic edges



a proper 4-coloring of G



a palette of 4 colors

Graph Coloring

- A central problem in graph theory and computer science.
- But the task of coloring a graph with a minimum number of colors is a notoriously hard problem.

Theorem [Feige and Killian '98, Zuckerman '07]

For any $\epsilon > 0$, it is NP-hard to approximate the number of colors needed to within a factor $n^{1-\epsilon}$.

Feige, U. and Kilian, J. (1998).
Zero knowledge and the chromatic number.
Journal of Computer and System Sciences, 57(2):187–199.

Zuckerman, D. (2007).
Linear degree extractors and the inapproximability of max clique and chromatic number.
Theory of Computing, 3(6):103–128.

Graph Coloring

- So in many applications, we instead focus on coloring a graph using a number of colors based on some graph parameter.
- An important and well-studied case: $(\Delta + 1)$ coloring
  - Δ : maximum degree

Every graph admits a $(\Delta + 1)$ coloring

- Iterate over the vertices in an arbitrary order
- Assign each vertex a color that is not in its neighborhood
- Since the max-degree is Δ, such a color always exists

The runtime of this algorithm is $O(m + n)$ time which is clearly optimal as we need this much time simply to read the input.

Sublinear Algorithms

- How about massive graphs? Is there an even more efficient algorithm?

Can we design sublinear algorithms for $(\Delta + 1)$-coloring problem in modern models of computation for processing massive graphs?

- This paper answer this fundamental question in the affirmative for modern models of computation.

Sublinear Algorithms

- Sublinear means sublinear in the number of edges. Since the output of $(\Delta + 1)$-coloring is always linear in the number of vertices.

Can compute a $(\Delta + 1)$-coloring by examining only a tiny fraction of the edges in a graph?

- Based on the computational models, we may want sublinear time, space, or communication algorithms.

Sublinear Algorithms

1. Sublinear time algorithms:
   - Process the graph faster than even reading the entire input.

2. Streaming algorithms:
   - Process the graph on the fly with limited memory.

3. Massively parallel computation (MPC) algorithms:
   - Process the graph in a distributed fashion with limited communication.

Main Results

Surprisingly, we can get highly efficient sublinear algorithms for $(\Delta + 1)$-coloring in three models of computation.

All algorithms are randomized and behave as follows:

- either output a valid $(\Delta + 1)$-coloring (w.h.p.), or
- output FAIL.

these algorithms **never output** an invalid coloring.

Result 1: Sublinear Time Algorithms

The standard query model for dense graphs:

- Degree queries: what is the degree of the vertex v?
- Pair queries: is (u, v) an edge?
- Neighbor queries: what is the k-th neighbor of the vertex v?

Prior Results:
No sublinear time algorithm for $(\Delta + 1)$ coloring.
Fastest algorithm: the $O(n^2)$ time greedy algorithm.

Result 1: Sublinear Time Algorithms

The standard query model for dense graphs:

- Degree queries: what is degree of the vertex v?
- Pair queries: is (u, v) an edge?
- Neighbor queries: what is the k-th neighbor of the vertex v?

Paper's Result:

An $\tilde{O}(n\sqrt{n})$ time algorithm for $(\Delta + 1)$ coloring.

- Queries are chosen non-adaptively(in parallel).
- $\Omega(n\sqrt{n})$ query lower bound even for adaptive algorithms.

Result 2: Streaming Algorithms

Semi-streaming algorithms:

- Edges are appearing one by one in a stream.
- Process the stream in one pass and $\tilde{O}(n)$ space.

Prior Results:

- No single-pass streaming algorithm for $(\Delta + 1)$ coloring with $o(n^2)$ space even in insertion-only streams.
- Parallel to this work. Easier problem of $(\Delta + o(\Delta))$: a semi-streaming algorithm using sulinear space by [Bera and Ghosh, 2018].

Result 2: Streaming Algorithms

Semi-streaming algorithms:

- Edges are appearing one by one in a stream.
- Process the stream in one pass and $\tilde{O}(n)$ space.

Paper's Result:

A single-pass $\tilde{O}(n)$ space streaming algorithm for $(\Delta + 1)$ coloring.

- $\Omega(n)$ space is clearly necessary for this problem(optimal).
- This algorithm works even in dynamic graph streams.

Result 3: MPC Algorithms

MPC algorithms with $\tilde{O}(n)$ memory per-machine:

- Edges are partitioned arbitrarily across multiple machines.
- Machines can send and receive $\tilde{O}(n)$ messages in synchronous rounds.

Prior Results:

- An $\mathcal{O}(\log\log\Delta \cdot \log^*(n))$ round algorithm with $\tilde{O}(n)$ memory per-machine for $(\Delta+1)$ coloring [Parter, 2018].
- Parallel to this work. The round-complexity improved to $\mathcal{O}(\log^*(n))$ rounds [Parter and Su, 2018].
- Easier problem of $(\Delta + o(\Delta))$ coloring: an $\mathcal{O}(1)$ round algorithm with $n^{1+\Omega(1)}$ memory [Harvey et al., 2018].

Result 3: MPC Algorithms

MPC algorithms with $\tilde{O}(n)$ memory per-machine:

- Edges are partitioned arbitrarily across multiple machines.
- Machines can send and receive $\tilde{O}(n)$ messages in synchronous rounds.

Paper's Result:

An $\mathcal{O}(1)$ round $\tilde{O}(n)$ memory MPC algorithm for $(\Delta + 1)$ coloring.

- This algorithm only requires one round assuming public randomness(optimal).
- The first constant round MPC algorithm with $\tilde{O}(n)$ memory for one of "classic four local distributed graph problems".

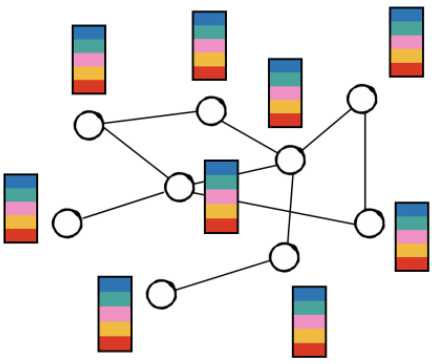
Palette Sparsification Theorem

How do we design these Sublinear Algorithms?

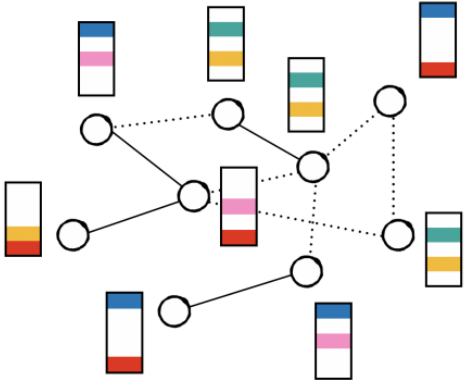
Palette Sparsification Theorem

Let $G(V, E)$ be any n-vertex graph and Δ be the maximum degree in G. Suppose for each vertex $v \in V$, we independently pick a set $L(v)$ of colors of size $\Theta(logn)$ uniformly at random from $\{1, \ldots, \Delta + 1\}$. Then with high probability there exists a proper coloring $C : V \mapsto \{1, \ldots, \Delta + 1\}$ of G such that for all vertices $v \in V$, $C(v) \in L(v)$.

Palette Sparsification Theorem: Illustration



(a) A graph $G = (V, E)$ with its original palettes of $(\Delta + 1)$ colors.



(b) The sampled palettes of the vertices and the resulting conflict graph – the dotted edges can be ignored now.

ColoringAlgorithm

ColoringAlgorithm(G, Δ) : A meta-algorithm for finding a $(\Delta + 1)$-coloring in a graph $G(V, E)$ with maximum degree Δ.

ColoringAlgorithm(G, Δ)

1. Sample $\Theta(\log n)$ colors $L(v)$ uniformly at random for each vertex $v \in V$ (as in Palette Sparsification Theorem).
2. Define, for each color $c \in \{1, \ldots, \Delta + 1\}$, a set $\chi_c \subseteq V$ where $v \in \chi_c$ iff $c \in L(v)$.
3. Define $E_{conflict}$ as the set of all edges (u, v) where both $u, v \in \chi_c$ for some $c \in \{1, \ldots, \Delta + 1\}$.
4. Construct the conflict graph $G_{conflict}(V, E_{conflict})$.
5. Find a proper list-coloring of $G_{conflict}(V, E_{conflict})$ with $L(v)$ being the color list of vertex $v \in V$.

Palette Sparsification Theorem: Properties

Lemma

In **ColoringAlgorithm**(G, Δ), *with high probability*, the output is a valid $(\Delta + 1)$-coloring of the graph G.

Proof.

- By Palette Sparsification Theorem, with high probability there does exist a valid list-coloring of $G_{conflict}$.
- Since $E_{conflict}$ contains all possible monochromatic edges that arise in any list-coloring of G, any valid list-coloring of $G_{conflict}$, if one exists, is a valid coloring of the input graph G and vice versa.
- So $(\Delta + 1)$-coloring of input graph G reduces to list-coloring of the graph $G_{conflict}$.

□

Palette Sparsification Theorem: Properties

Lemma

In **ColoringAlgorithm**(G, Δ), *with high probability*, the maximum degree in graph $G_{conflict}$ is $O(\log^2 n)$ and thus $G_{conflict}$ has $O(n\log^2 n)$ edges.

Proof.

- Fix any vertex $v \in V$ and its list of colors $L(v)$
- Let K be the number of colors sampled by each vertex.
- For any neighbor u of v in G, let $X_u \in \{0, 1\}$ be an indicator random variable which is 1 if and only if v is a neighbor of u in $G_{conflict}$
- For u to be a neighbor of v in $G_{conflict}$, $L(u) \cap L(v)$ must be non-empty. Thus:

$$\Pr(X_u = 1) = \Pr\left(\bigcup_{c \in L(v)} (c \in L(u))\right) \leq \sum_{c \in L(v)} \Pr(c \in L(u)) = K \cdot \frac{K}{\Delta + 1} = \frac{K^2}{\Delta + 1}$$

Propostion(Chernoff Bound)

Suppose $X_1, \ldots, X_n$ are independent random variables taking values in $[0, 1]$, and let $X = \sum_{i=1}^{n} X_i$. Then, for any $\epsilon > 0$,

$$\Pr\left(|X - \mathbb{E}[X]| \geq \epsilon \cdot \mathbb{E}[X]\right) \leq 2 \cdot \exp\left(\frac{-\epsilon^2 \cdot \mathbb{E}[X]}{2 + \epsilon}\right).$$

Proof(Continued).

Moreover, $\deg_{G_{conflict}}(v) = \sum_{u \in N_G(v)} X_u$ and thus:

$$\mathbb{E}[\deg_{G_{conflict}}(v)] \leq \Delta \cdot \frac{K^2}{\Delta + 1} = O(\log^2 n),$$

As $\deg_{G_{conflict}}(v)$ is the sum of independent 0-1 random variables X_u for $u \in N_G(v)$, we can apply the Chernoff bound (with $\epsilon = 10$) and obtain:

$$\Pr\left(\deg_{G_{conflict}}(v) \geq 10 \cdot \mathbb{E}[\deg_{G_{conflict}}(v)]\right) \leq 2 \cdot \exp\left(\frac{-100 \log^2 n}{12}\right) < \frac{1}{n^8}. \quad \square$$

Palette Sparsification Theorem: Results

- Construction of $G_{conflict}$ is non-adaptive.
- The graph $G_{conflict}$ is very sparse.

Palette Sparsification Theorem thus allows non-adaptive sparsification of a graph with $O(n\Delta)$ edges to a graph with $\tilde{O}(n)$ while preserving $(\Delta+1)$-colorability with high probability.

A One-Pass $\tilde{O}(n)$ Space Streaming Algorithm

- At the start, each vertex v samples $\Theta(\log n)$ colors independently and uniformly at random – let $L(v)$ be the set of colors sampled by vertex v.
- When an edge (u, v) arrives in the stream, we now determine its membership in the conflict graph by a simple test: if $L(u) \cap L(v) \neq \emptyset$, add (u, v) to E_{conflict}.
- At the end of the stream, we list-color the graph $G_{\text{conflict}}(V, E_{\text{conflict}})$.

A One-Pass $\tilde{O}(n)$ Space Streaming Algorithm

- Total space used by algorithm is $\tilde{O}(n)$:
  - space to store the color lists $L(v)$, which is a total of $\tilde{O}(n)$ space, and
  - space to store edges in E_{conflict}, also $\tilde{O}(n)$ space.

- We can maintain the set E_{conflict} in $\tilde{O}(n)$ space even for dynamic streams where the graph is revealed by an arbitrary sequence of edge insertions and deletions.

Maintaining Algorithm For Dynamic Streams

Theorem

There exists a randomized single-pass semi-streaming algorithm that given a graph G with maximum degree Δ presented in a dynamic stream, with high probability finds a $(\Delta+1)$ coloring of G with using $\tilde{O}(n)$ space and polynomial time.

Maintaining Algorithm For Dynamic Streams

Proposition: *There exists an streaming algorithm that given a subset $P \subseteq V \times V$ of pairs of vertices and an integer $k \geq 1$ at the beginning of a dynamic stream, outputs with high probability a set S of k edges from the edges in P that appear in the final graph (it outputs all edges if their number is smaller than k). The set S of edges can be either chosen uniformly at random with replacement or without replacement. The space of algorithm is $O(k \cdot log^3 n)$.*

Maintaining Algorithm For Dynamic Streams

Lemma

$G_{\text{conflict}}(V, E_{\text{conflict}})$ can be constructed in $\tilde{O}(n)$ space and polynomial time in dynamic streams with high probability.

Proof

- We construct the sets $\chi_1, \ldots, \chi_{\Delta+1}$ and store the sets in $O(n \log n)$ space. For any vertex $v \in V$, we define the set P_v of all edge slots between v and $\bigcup_{c \in L(v)} \chi_c$, i.e., all vertices that may have an edge to v in E_{conflict}, and run the algorithm in Proposition with $P = P_v$ and parameter $k = O(\log^2 n)$.
- As we conditioned earlier, the degree of each vertex is at most k in G_{conflict}. Hence, by Proposition, with high probability, we find all neighbors of this vertex in G_{conflict}. Taking a union bound over all vertices in V, with high probability, we can construct the graph G_{conflict}. As all these steps can be implemented in polynomial time, we obtain the final result. □

Maintaining Algorithm For Dynamic Streams

- These observations are already enough to achieve a semi-streaming algorithm with exponential-time.
- we can simply use an exponential time algorithm to find a proper list-coloring of G_{conflict} which would be a $(\Delta + 1)$ coloring of G by sparsification.
- In the following, we focus on designing a polynomial-time algorithm. The list-coloring of G_{conflict} follows the same approach as the proof of the palette sparsification theorem; hence, we omit the details.

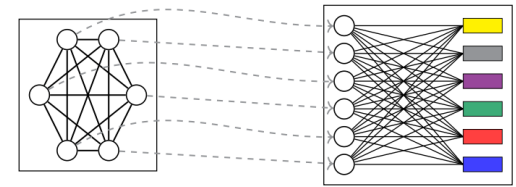
Proof of the Palette-Sparsification Theorem

Palette-Sparsification Theorem

Let $G = (V, E)$ be any graph with n vertices and maximum degree Δ. Suppose for every vertex $v \in V$, we sample a list $L(v)$ of $s = \theta(\log n)$ colors independently and uniformly at random from $\{1, \ldots, \Delta + 1\}$ (with repetition). Then, with high probability, G can be colored by coloring every $v \in V$ from the list $L(v)$; in other words, G is list-colorable from $\{L(v) | v \in V\}$.

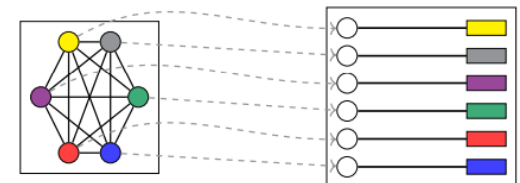
Warm up: Special case

- Imagine if the graph we were working with were a clique
- We can map the original graph to a Palette Graph



Original Graph Palette Graph

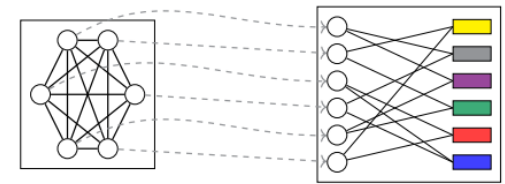
Warm up: Special case



- The problem of finding a $(\Delta + 1)$ coloring in the original graph is equivalent to finding a perfect matching in the palette graph

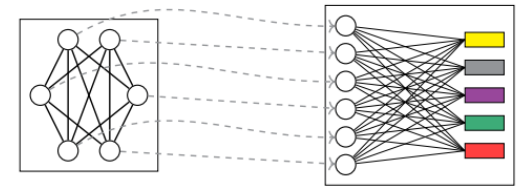
Warm up: Special case

- Relation with the Palette sparsification theorem: Random subgraphs of the palette graph of a clique contain a perfect matching (a result known from random graph theory)



Warm up: A more general case

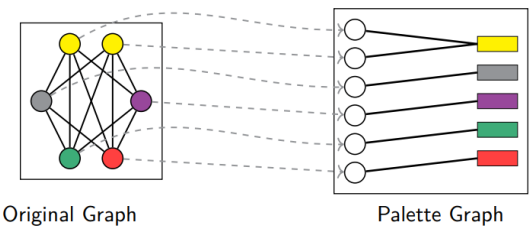
- Coloring a clique minus a perfect matching
- Suppose we have a clique and we remove a perfect matching from it



- $(\Delta + 1)$ coloring is equivalent to finding a good subgraph in the palette graph

Warm up: A more general case

- good: When one color is assigned to more than one node, these nodes form an independent set in the original graph



- Relation to the Palette sparsification theorem: Random subgraphs of the palette graph of a clique minus a perfect matching contain a good subgraph

A general method for almost cliques

- When our graphs are close to being a clique:

- Find a subgraph of the palette graph:
- Degree exactly one for vertices on left.
- Neighbors of vertices on right can only be an independent set in the original graph.
- Palette sparsification theorem reduces to a random graph theory question

- not that helpful for graphs that are far from cliques

Graphs that are far from cliques

- We will consider low degree graphs, like a graph where all vertices have degree less than or equal to $\Delta/2$
- (Every vertex except for one, has degree $<= \Delta/2$)

Low degree graphs

- Our coloring strategy:

  1. Pick a color uniformly at random from $\{1, \ldots, \Delta + 1\}$ for all uncolored vertices.
  2. Assign the color to each vertex if it is not assigned to its neighbors in this iteration or previous ones.
  3. Repeat until all vertices are colored.

- Every vertex has constant probability of being colored in each iteration
  (In the worst case we have $\Delta/2$ neighbors of different colors, so this constant is at least $1/2$)
- After O(log n) iterations, all vertices are colored

Why O(log n) iterations?

- Let's say at the start of an iteration, there are k uncolored vertices
- Each uncolored vertex gets colored independently with probability at least c (constant)
- So after one iteration, the expected number of uncolored vertices is at most $(1 - c) * k$
- After t iterations, expected uncolored vertices $<= (1 - c)^t * n$

Why O(log n) iterations?

- We want this to be less than 1 to ensure all vertices are colored with high probability:

$$(1-c)^t \cdot n < 1$$

Taking logarithms on both sides:

$$t \log(1-c) + \log n < 0$$

$$t > \frac{-\log n}{\log(1-c)}$$

Since c is a constant, $\log(1-c)$ is also constant (and negative), so $\frac{-1}{\log(1-c)}$ is a constant. Therefore:

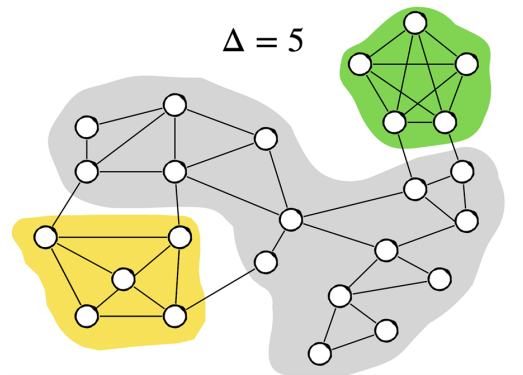
$$t = O(\log n)$$

General proof

- We have seen proofs for extreme cases, but what about the general case of a graph?
- Neither strategy works for the other case
- The strategy for coloring low degree graphs provably requires $\Omega(\Delta)$ iterations on a clique
- The strategy for coloring a clique is not appropriate for low-degree graphs since they lack structure

High Level Idea

- We can decompose a given graph into dense and sparse sections



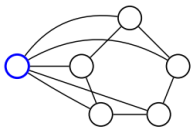
Decomposition of A Graph

- We use a simple extension of the decomposition of Harris, Schneider, and Su [Harris et al., 2016] for distributed $(\Delta + 1)$ coloring

- Extended HSS Decomposition: For any $\epsilon \in (0, 1)$, any graph $G(V, E)$ can be decomposed into:
- Sparse vertices: The neighborhood of each sparse vertex is missing at least $\varepsilon \cdot \binom{\Delta}{2}$ edges.
- A collection of almost-cliques: Each almost-clique C:
  - contains $(1 \pm \varepsilon)\Delta$ vertices.
  - every vertex in C has $\leq \varepsilon\Delta$ neighbors outside C.
  - every vertex in C has $\leq \varepsilon\Delta$ non-neighbors inside C.

Sparse Vertices

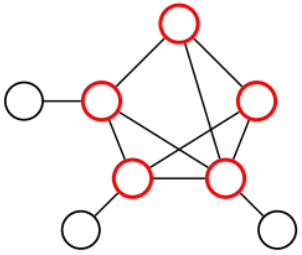
- A sparse vertex is a vertex who's neighborhood is ϵ far from being a clique
- If we look at the subgraph formed by all the neighbors of that vertex, it is missing ϵ fraction of edges of a clique



- With this description:
- Any low degree vertex is automatically a sparse vertex (Since we don't have enough edges to begin with)
- Some vertices that have a lot of edges connected to them may still be sparse (In the case where their neighborhood doesn't have any edges)

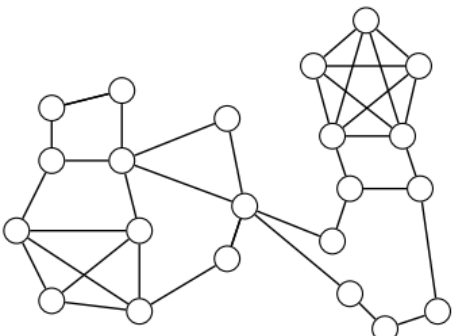
Almost Cliques

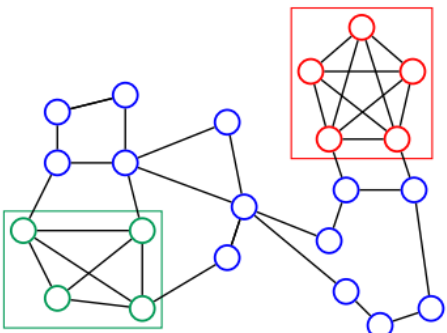
- We can have multiple almost cliques



Proof Strategy

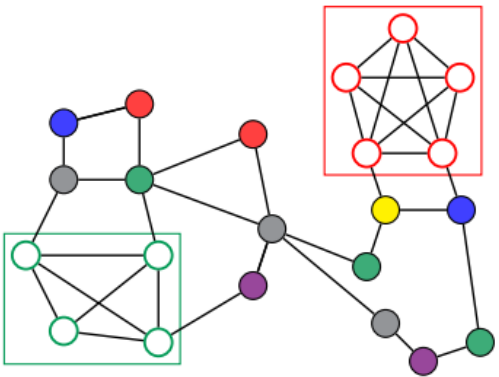
1. Fix an extended HSS decomposition of the graph for $\varepsilon \approx 0.001$





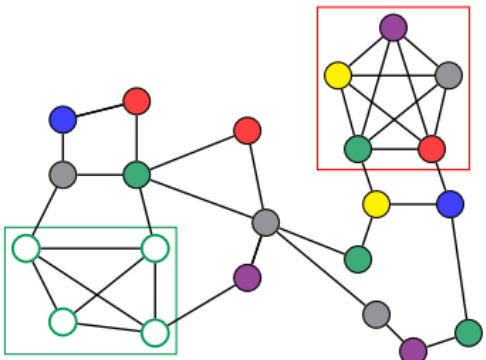
Proof Strategy

2. Use the first half of colors in $L(\cdot)$ to color sparse vertices



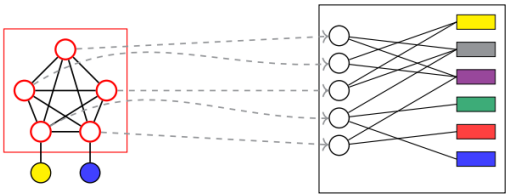
Proof Strategy

3. Iterate over the almost-cliques one by one and color each one using the remaining half of $L(\cdot)$



The main challenges

- The size of the almost clique can be very large $O(1 + \epsilon\Delta)$
- The palette graph is not a complete bipartite graph since vertices in an almost-clique may have some colored neighbors outside



Extensions of the Palette Sparsification Theorem

DISTRIBUTED $(\Delta + 1)$-COLORING IN SUBLOGARITHMIC ROUNDS

- Goal: $(\Delta + 1)$-**coloring** in LOCAL model.
- Runtime:

$$O\left(\sqrt{\log \Delta}\right) + 2^{O\left(\sqrt{\log \log n}\right)}$$

- **Step 1: Classify Nodes**
  - Dense: neighbors are mostly connected (almost clique).
  - Sparse: not dense.
  - Done locally in $O(1)$ rounds.
- **Step 2: Handle Dense Components**
  - Each dense part has weak diameter ≤ 2.
  - Elect a leader in 2 rounds to assign colors centrally.
  - Most dense nodes are colored in one coordinated step.

Shrinking, Finishing, and Final Runtime

- **Shrink Dense Parts**:
  - Each round reduces size by $1 - \Theta(\sqrt{\delta})$
  - After $\lceil \sqrt{\log \Delta} \rceil$ rounds: $\tilde{O}(1)$-size
- **Sparse Remainder**:
  - Each sparse node has $\Omega(\varepsilon^2 \Delta)$ colors and only $O(\log n)$ live neighbors.
  - Elkin–Pettie–Su list coloring in:

$$O(\log(1/\varepsilon)) + 2^{O(\sqrt{\log \log n})}$$

- **Tiny Residual Graph**:
  - Degree is $O(\log n) \cdot 2^{O(\sqrt{\log \Delta})}$
  - Final coloring using Barenboim–Elkin:

$$O(\sqrt{\log \Delta}) + 2^{O(\sqrt{\log \log n})}$$

Palette Sparsification with Fewer Colors

Could we establish palette sparsification theorem by sampling only $O(1)$ colors at each vertex?

Claim: There exist graphs such that if each vertex samples $o(\log n)$ colors, then almost certainly no feasible coloring exists among sampled colors.

- Suppose G consists of $n/\log n$ copies of the graph $K_{\log n}$.
- Then if each vertex samples $o(\log n)$ colors, then almost certainly some clique fails to sample $\log n$ distinct colors.
- So almost certainly no valid coloring exists among sampled colors.

Very Recent Work

In fact, if we want a vanishingly small failure probability, we need to sample more than $\log n$ colors at each vertex.

How big does the constant hidden in $O(\log n)$ need to be?

Asymptotically Optimal Version [Kahn and Kenney '23]

For any $\varepsilon > 0$, if each vertex in a graph G independently samples $(1+\varepsilon)\log n$ colors uniformly at random from $\{1, 2, \ldots, \Delta + 1\}$, then w.h.p. there is a valid coloring of the graph G such that each vertex is assigned one of its sampled colors.

(Degree + 1) Coloring

The greedy coloring algorithm for $(\Delta + 1)$-coloring also works if we restrict each vertex v to only use colors from an arbitrary subset of $(\deg(v) + 1)$ colors.

(Degree + 1) Palette Sparsification

[Assadi and Alon '20]

Palette sparsification theorem holds even if each vertex v samples $O(\log n)$ colors from $\{1, 2, \ldots, \deg(v) + 1\}$.

[Halldorsson, Kuhn, Nolin, Tonoyan '22]
Extend it to arbitrary subsets of (degree + 1) colors.

Δ-Coloring in Streams

Brooks theorem: All graphs other than cliques and odd cycles are Δ-colorable.

Streaming Brooks Theorem [Assadi, Kumar, and Mittal '22]
There is a $\tilde{O}(n)$ space single-pass streaming algorithm that can color any Δ-colorable graph with Δ colors.

Palette Sparsification for C-Coloring

Does a similar sparsification result hold for C-colorable graphs for $C \leq \Delta$?

Claim: There exist Δ-colorable graphs such that unless each vertex samples $\Omega(\Delta^{0.5})$ colors, w.h.p. there is no feasible coloring exists among the sampled colors.

- Consider the graph $K_{\Delta+1}$ and remove an edge between any pair u, v of vertices — this is a Δ-colorable graph.
- In any Δ-coloring, u and v must receive the same color.
- But unless each of u and v samples at least $\Omega(\Delta^{0.5})$ colors, it is unlikely they have a common color.

Other Extensions

Sublinear algorithms for coloring in terms of other natural parameters.

A graph G has degeneracy κ if every subgraph of G has a vertex of degree at most κ. The parameter κ is always at most Δ but could be much smaller.

[Bera, Chakrabarti, Ghosh'19]

- Sublinear algorithms for coloring with $\kappa + o(\kappa)$ colors.
- In contrast to $(\Delta + 1)$-coloring, the problem of finding a $(\kappa + 1)$-coloring turns out to be provably hard for both sublinear space and time algorithms.

Concluding Remarks

- A non-adaptive sparsification result for $(\Delta + 1)$-coloring.
- Any graph can be sparsified to a graph with $\tilde{O}(n)$ edges such that list-coloring the sparsified graph is equivalent to $(\Delta + 1)$-coloring the original graph.
- The sparsification yields essentially optimal sublinear algorithms for three well-studied models of sublinear algorithms:
  - Streaming model for sublinear space,
  - Query model for sublinear time, and
  - MPC model for sublinear communication.

References

Feige, U. and Kilian, J. (1998).
Zero knowledge and the chromatic number.
Journal of Computer and System Sciences, 57(2):187–199.

Zuckerman, D. (2007).
Linear degree extractors and the inapproximability of max clique and chromatic number.
Theory of Computing, 3(6):103–128.

Thank you!